Dossier Technique

« RescueAid »

Préparé par :

- Chebbi Sirine
- Taouai Maryem
- Zarai Eya
- Dridi Tassnim
- Younes Mezen
- Meddeb Moemen

2026 /2027

Table des matières

1 Contexte et problématique 4
1.1 Contexte 4
1.2 Problématique 4
1.3 Solution proposée : RescueAid 4
2 Acteurs et cas d'utilisation général 4
1.4 Acteurs 4
1.5 Cas d'utilisation généraux 4
3 Fonctionnalités principales 5
4 Architecture globale 5
1.6 Vue d'ensemble 5
1.7 Principes d'architecture 5
5 Découpage en microservices 6
6 Données gérées par chaque service 6
1.8 MS1 – User Service 6
1.9 MS2 – Incident & Needs Service 6
1.10 MS3 – Location & Zone Service 6
1.11 MS4 – Donation & Campaign Service 6
1.12 MS5 – Resource & Distribution Service 6
1.13 MS6 – Association Service 7
1.14 MS7 – Transport Service 7
7 Répartition des Tâches et Responsabilités de l'Équipe 7
8 Interactions entre les services 8
1.15 Communications synchrone 8
1.16 Scénario complet 8
9 Contrats d'API REST et Spécification des Événements Asynchrones 8
1.17 MS1 – User Service (Commun - Port 8081) 8
1.18 MS2 – Incident & Needs Service (Port 8082) 9
1.19 MS3 – Location & Zone Service (Port 8083) 9
1.20 MS4 – Donation & Campaign Service (Port 8084) 9
1.21 MS5 – Resource & Distribution Service (Port 8085) 10
1.22 MS6 – Association Service (Port 8086) 10
1.23 MS7 – Transport Service (Port 8087) 10
10 Conception 11
1.24 Diagramme des cas d'utilisations 11
1.25 Diagrammes de classes 12

10.1.1 Diagramme de classe globale....12
10.1.2 Diagramme de classe : MS1-User Service....12
10.1.3 Diagramme de classe : MS2-Icident & Needs Service....13
10.1.4 Diagramme de classe : MS3- Location & Zone Service....13
10.1.5 Di agramme de classe : MS4- Donation & Campaign Service....14
10.1.6 Diagramme de classe : MS5- Resource & Distribution Service....15
10.1.7 Diagramme de classe : MS6- Association Service....16
10.1.8 Diagramme de classe : MS7- Transport Service....16
11 Conclusion....17

1 Contexte et problématique

1.1 Contexte

Lors d'une catastrophe (inondation, incendie, séisme), l'information est dispersée : les citoyens signalent les incidents par téléphone ou sur les réseaux sociaux, les associations ignorent où intervenir, les dons et les stocks sont gérés séparément et personne n'a une vision globale. Cette désorganisation retarde les secours.

1.2 Problématique

Comment coordonner rapidement les signalements, les besoins des victimes, les dons, les ressources, les associations, les centres de transport et les zones géographiques concernées, tout en gardant un système évolutif, résilient et maintenable ?

1.3 Solution proposée : RescueAid

RescueAid est une plateforme de gestion des urgences et de coordination de l'aide humanitaire. Elle repose sur 7 microservices Spring Boot indépendants, exposant des API REST documentées avec Swagger et testables avec Postman. Elle combine deux objectifs :

• Gestion des urgences : signalement, localisation, priorisation des incidents et identification des besoins.

• Gestion de l'aide : campagnes de dons, stocks, distributions, mobilisation des associations et logistique de transport.

2 Acteurs et cas d'utilisation général

1.4 Acteurs

Acteur	Rôle
Citoyen	Signale un incident, décrit les besoins, suit l'état de son signalement et consulte les campagnes
Donateur	Consulte les campagnes de dons et enregistre ses dons
Association	Gère son profil, ses spécialités, sa disponibilité et ses interventions
Administrateur	Supervise les incidents, valide les campagnes, gère les stocks, les zones et le parc de transport

1.5 Cas d'utilisation généraux

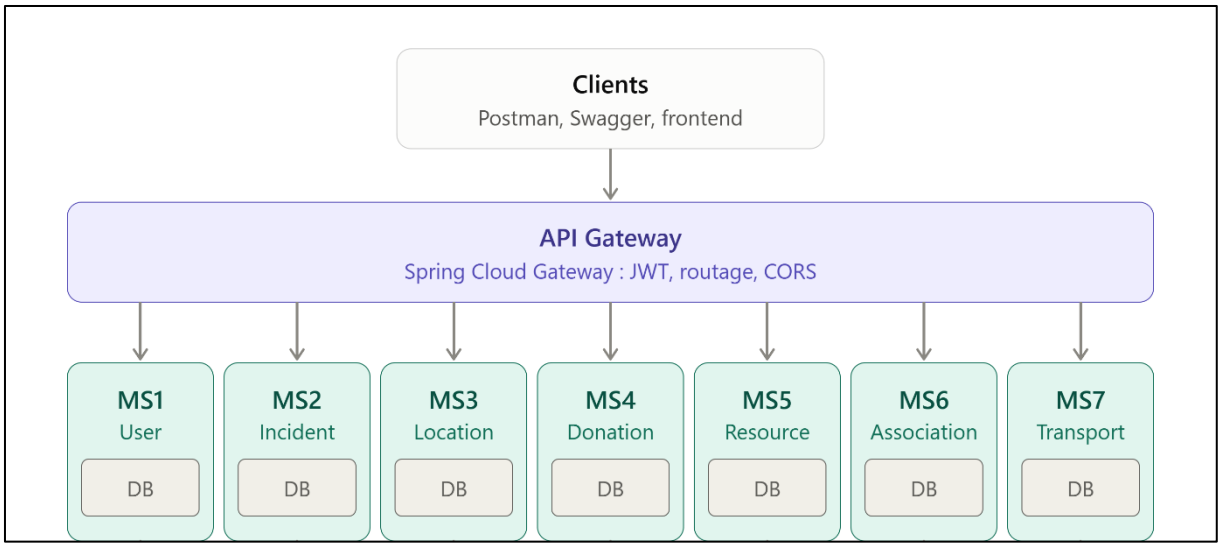
- S'inscrire, s'authentifier et gérer son profil (tous les acteurs).
- Signaler un incident et ses besoins (citoyen).
- Consulter les incidents proches et les zones à risque (administrateur, association).
- Consulter les campagnes et faire un don (donateur, citoyen).
- Valider ou clôturer une campagne (administrateur).
- Gérer les stocks et les distributions (administrateur).
- Rechercher les associations adaptées à un incident (administrateur).
- Planifier un trajet de transport (administrateur).

3 Fonctionnalités principales

- Authentification JWT et contrôle d'accès par rôles (RBAC).
- Signalement d'incidents et de besoins avec calcul automatique d'un score de priorité.
- Gestion des zones, des points d'intervention, recherche des incidents à proximité et détection des zones à risque.
- Gestion des dons et des campagnes, avec proposition automatique de campagne selon les besoins déclarés.
- Gestion des stocks, détection des pénuries et proposition d'allocation des ressources.
- Gestion des associations et recommandation des plus adaptées à un incident.
- Gestion du parc de véhicules, des chauffeurs et des trajets, avec proposition du véhicule le plus adapté.

4 Architecture globale

1.6 Vue d'ensemble



1.7 Principes d'architecture

1. Une base de données (ou un schéma isolé) par microservice : aucun accès direct à la base d'un autre service.
2. Aucun partage d'entités Java : les services échangent des contrats d'API ou des événements.
3. Les références entre services sont de simples identifiants (incidentId, associationId, userId), sans relation JPA.
4. Chaque service conserve localement les données dont il a besoin, alimentées par les événements (copie locale ou read model).
5. Un service doit continuer à fonctionner si un autre est momentanément indisponible.
6. Communication REST synchrone limitée aux besoins clients ; événements asynchrones entre services.

5 Découpage en microservices

Microservices	Responsabilité	Port
MS1 – User Service (commun)	Comptes, authentification, rôles	8081
MS2 – Incident & Needs	Incidents, besoins, priorité	8082
MS3 – Location & Zone	Zones, points d'intervention	8083
MS4 – Donation & Campaign	Dons et campagnes	8084
MS5 – Resource & Distribution	Ressources, stocks, distributions	8085
MS6 – Association	Associations, spécialités, disponibilités	8086
MS7 – Transport	Véhicules, chauffeurs, trajets	8087

6 Données gérées par chaque service

1.8 MS1 – User Service

- **User** : id, nom, prénom, email, téléphone, motDePasse, rôle, actif, dateCréation, dateDernièreConnexion.

1.9 MS2 – Incident & Needs Service

- **Incident** : id, titre, description, type, gravité, nombreVictimes, adresse, latitude, longitude, statut, scorePriorité, reporterId, dateCréation, dateMiseAJour.
- **Need** : id, incidentId, type, quantitéDemandée, quantitéSatisfaite, unité, statut.

1.10 MS3 – Location & Zone Service

- **Zone** : id, nom, ville, coordonneesCentre, rayonKm, niveauRisque.
- **InterventionPoint** : id, zoneId, nom, type, coordonnees, capacitéMax, occupationActuelle, statut.
- **IncidentLocation** : incidentId, coordonnees, priorité, dateSynchro.

1.11 MS4 – Donation & Campaign Service

- **Campaign** : id, titre, description, incidentId, createurId, objectifMontant, montantCollecté, objectifKits, kitsCollectés, statut, dateDébut, dateFin.
- **Donation** : id, campaignId, donorId, typeDon, montant, nombreKits, modePaiement, date, statut.

1.12 MS5 – Resource & Distribution Service

- **Resource** : id, nom, catégorie, unité.
- **Stock** : id, resourceId, entrepotId, quantitéDisponible, seuilAlerte.
- **Distribution** : id, incidentId, resourceId, entrepotId, quantité, statut, datePlanifiée.

1.13 MS6 – Association Service

- **Association** : id, nom, adresse, téléphone, emailContact, latitude, longitude, rayonActionKm, capacitéIntervention, disponible, userId.
- **Speciality** : id, associationId, type.
- **Availability** : id, associationId, dateDébut, dateFin.
- **Intervention** : id, associationId, incidentId, statut, dateAffectation.

1.14 MS7 – Transport Service

- **Vehicle** : id, immatriculation, type, capacitéKg, capacitéPersonnes, statut, ville, latitude, longitude, dernièreMiseAJourGps.
- **Driver** : id, nom, téléphone, numéroPermis, disponible, vehicleId.
- **Trip** : id, vehicleId, driverId, incidentId, distributionId, pointDépart, destination, typeTrajet, chargeKg, nombrePersonnes, dateDépart, dateArrivée, statut.

7 Répartition des Tâches et Responsabilités de l'Équipe

Conformément au guide de validation, l'équipe se compose de 6 étudiants. **MS1 – User Service** constitue le microservice commun développé conjointement par l'ensemble du groupe.

Membre de l'Équipe	Microservice Attribué	Rôle dans le Projet
Younes Mezen	**MS2 – Incident & Needs Service** (Port 8082)	Saisie des signalements, algorithme de score de priorité et gestion des besoins.
Taouai Maryem	**MS3 – Location & Zone Service** (Port 8083)	Modélisation des zones de risque, points d'intervention SIG et indexation spatiale.
Dridi Tassnim	**MS4 – Donation & Campaign Service** (Port 8084)	Gestion des campagnes de dons, collecte financière/kits et validation admin.
Meddeb Moemen	**MS5 – Resource & Distribution Service** (Port 8085)	Suivi des stocks matériels, détection automatique des pénuries et allocations.
Chebbi Sirine	**MS6 – Association Service** (Port 8086)	Registre des associations, spécialités et algorithme de recommandation d'équipe.
Zarai Eya	**MS7 – Transport Service** (Port 8087)	Logistique de la flotte, gestion des chauffeurs et routage des véhicules.
Toute l'équipe	**MS1 – User Service (Commun)** (Port 8081)	Développement collectif de l'authentification JWT, RBAC et gestion des profils.

8 Interactions entre les services

1.15 Communications synchrone

- Client → API Gateway → microservice (REST/JSON).
- Le JWT émis par MS1 est vérifié localement par la gateway et par chaque service avec la même clé de signature : aucun appel à MS1 à chaque requête.
- Les appels REST directs entre microservices sont évités. S'ils sont nécessaires, ils utilisent OpenFeign avec timeout, retry et circuit breaker (Resilience4j).
- Exemple : l'administrateur appelle GET /api/vehicles/suggestions sur MS7 via la gateway pour choisir un véhicule.

1.16 Scénario complet

1. Un citoyen s'authentifie (MS1) et reçoit un JWT.
2. Il signale une inondation (MS2), qui calcule le score de priorité et enregistre les besoins.
3. MS3 lit l'incident dans MS2, enregistre sa localisation, recherche les incidents proches et détecte une éventuelle zone à risque.
4. MS6 lit l'incident dans MS2 et calcule les associations recommandées à partir de ses propres données.
5. MS4 lit les besoins dans MS2 et propose une campagne ; l'administrateur la valide.
6. MS5 compare les besoins aux stocks, détecte les pénuries et propose une allocation.
7. MS7 propose un véhicule adapté et planifie le trajet de livraison.

9 Contrats d'API REST et Spécification des Événements Asynchrones

1.17 MS1 – User Service (Commun - Port 8081)

- **Endpoints REST Principaux** :
  - POST /api/v1/auth/register : Création d'un nouveau compte utilisateur. (201 Created, 400 Bad Request, 409 Conflict)
  - POST /api/v1/auth/login : Authentification et émission du jeton JWT. (200 OK, 401 Unauthorized)
  - GET /api/v1/users/{id} : Consultation du profil utilisateur. (200 OK, 404 Not Found)
  - PUT /api/v1/users/{id} : Mise à jour des informations de profil. (200 OK, 400 Bad Request, 403 Forbidden)
- **Événements Asynchrones** :
  - *Publié* : UserRegisteredEvent (id, email, role).

1.18MS2 – Incident & Needs Service (Port 8082)

- **Endpoints REST Principaux** :
  - POST /api/v1/incidents : Signalement d'un incident avec ses besoins. (201 Created, 400 Bad Request)
  - GET /api/v1/incidents : Consultation de la liste des incidents avec filtres de gravité/statut. (200 OK)
  - GET /api/v1/incidents/{id} : Détail d'un incident et de ses besoins déclarés. (200 OK, 404 Not Found)
  - PUT /api/v1/incidents/{id}/status : Modification du statut de l'incident. (200 OK, 400 Bad Request)
- **Événements Asynchrones** :
  - *Publié* : IncidentReportedEvent (incidentId, latitude, longitude, scorePriorité, nombreVictimes, statut).
  - *Publié* : IncidentStatusChangedEvent (incidentId, nouveauStatut).

1.19 MS3 – Location & Zone Service (Port 8083)

- **Endpoints REST Principaux** :
  - POST /api/v1/zones : Création d'une zone géographique à risque. (201 Created, 400 Bad Request)
  - GET /api/v1/zones/risk-check : Détection si une coordonnée GPS se trouve dans une zone à risque (ST_Contains). (200 OK)
  - POST /api/v1/points : Ajout d'un point d'intervention (Hôpital, Abri, Entrepôt). (201 Created, 400 Bad Request)
  - GET /api/v1/points/nearest : Recherche du point d'intervention le plus proche d'un incident via ST_Distance_Sphere. (200 OK, 404 Not Found)
- **Événements Asynchrones** :
  - *Écouté* : IncidentReportedEvent (Insère ou met à jour la table IncidentLocation pour les calculs spatiaux).

1.20 MS4 – Donation & Campaign Service (Port 8084)

- **Endpoints REST Principaux** :
  - POST /api/v1/campaigns : Déclaration d'une nouvelle campagne de dons. (201 Created, 400 Bad Request)
  - GET /api/v1/campaigns/active : Liste des campagnes ouvertes aux dons. (200 OK)
  - POST /api/v1/donations : Enregistrement d'un don par un donateur. (201 Created, 400 Bad Request, 404 Not Found)
  - PUT /api/v1/campaigns/{id}/validate : Validation d'une campagne par l'administrateur. (200 OK, 403 Forbidden)
- **Événements Asynchrones** :
  - *Écouté* : IncidentReportedEvent (Évalue la nécessité d'ouvrir automatiquement une campagne de don).
  - *Publié* : DonationReceivedEvent (campaignId, donorId, montant).

1.21 MS5 – Resource & Distribution Service (Port 8085)

- **Endpoints REST Principaux** :
  - GET /api/v1/stocks : Inventaire des ressources matérielles disponibles par entrepôt. (200 OK)
  - POST /api/v1/stocks/replenish : Réapprovisionnement d'un stock d'urgence. (200 OK, 400 Bad Request)
  - POST /api/v1/distributions : Planification d'une distribution de biens vers un incident. (201 Created, 409 Conflict)
  - GET /api/v1/stocks/shortages : Détection des pénuries imminentes. (200 OK)
- **Événements Asynchrones** :
  - *Écouté* : IncidentReportedEvent (Analyse des besoins matériels par rapport aux stocks).
  - *Publié* : StockDepletedEvent (resourceId, entrepôtId, quantitéRestante).

1.22 MS6 – Association Service (Port 8086)

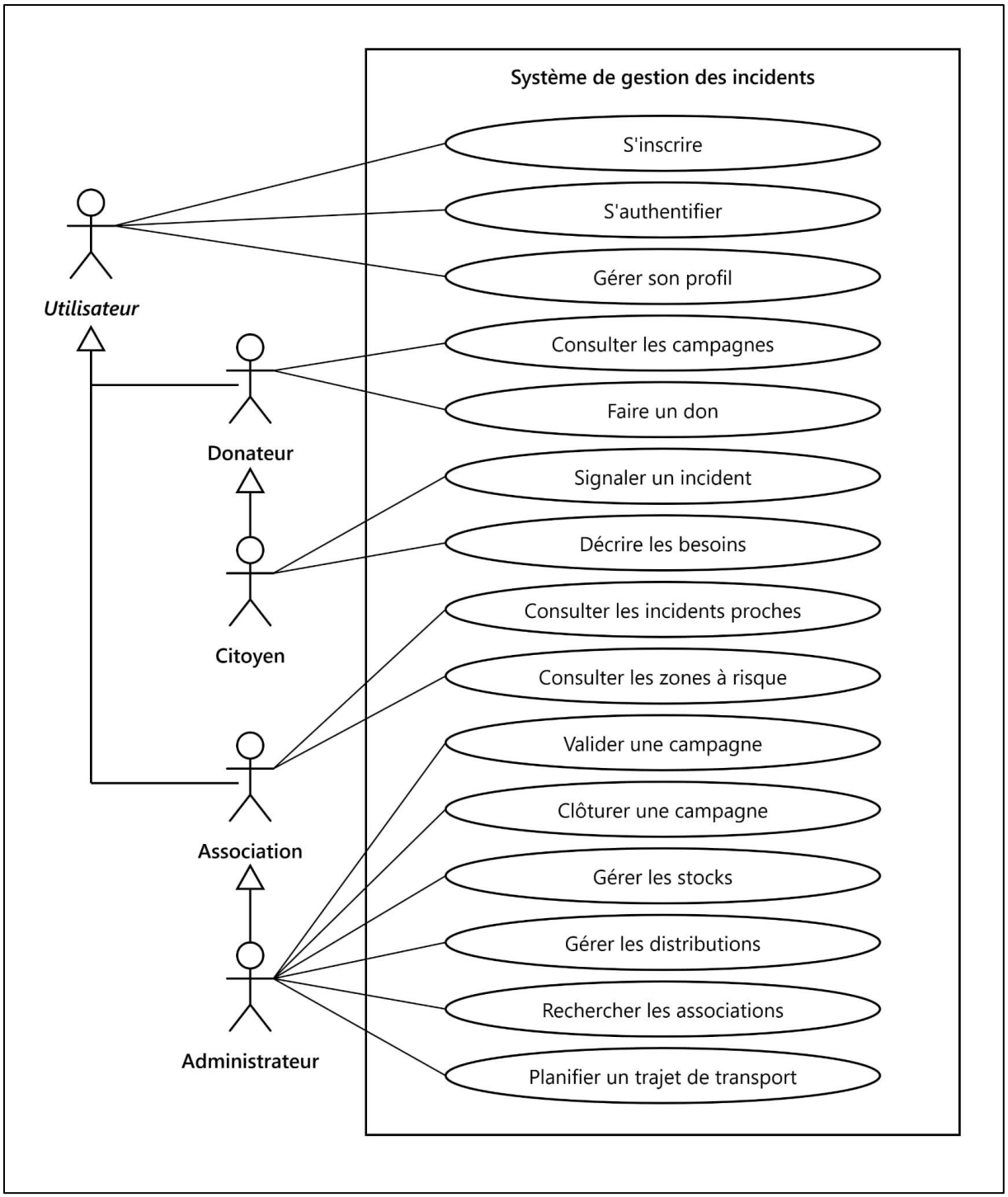
- **Endpoints REST Principaux** :
  - POST /api/v1/associations : Enregistrement d'une association humanitaire. (201 Created, 400 Bad Request)
  - GET /api/v1/associations/recommendations : Algorithme de recommandation des meilleures associations selon le type d'incident. (200 OK, 404 Not Found)
  - PUT /api/v1/associations/{id}/availability : Mise à jour de la plage de disponibilité d'une équipe. (200 OK, 400 Bad Request)
- **Événements Asynchrones** :
  - *Écouté* : IncidentReportedEvent (Déclenche le pré-calcul des équipes d'intervention compatibles).

1.23 MS7 – Transport Service (Port 8087)

- **Endpoints REST Principaux** :
  - POST /api/v1/vehicles : Enregistrement d'un véhicule dans le parc logistique. (201 Created, 400 Bad Request)
  - GET /api/v1/vehicles/suggestions : Proposition automatique du véhicule et du chauffeur les plus adaptés. (200 OK, 404 Not Found)
  - POST /api/v1/trips : Création et affectation d'une mission de transport. (201 Created, 400 Bad Request)
  - PUT /api/v1/trips/{id}/status : Suivi de la progression du trajet (IN_PROGRESS, COMPLETED). (200 OK, 404 Not Found)
- **Événements Asynchrones** :
  - *Écouté* : DistributionPlannedEvent (Déclenche la suggestion automatique d'un véhicule de livraison).
  - *Publié* : TripCompletedEvent (tripId, vehicleId, incidentId).

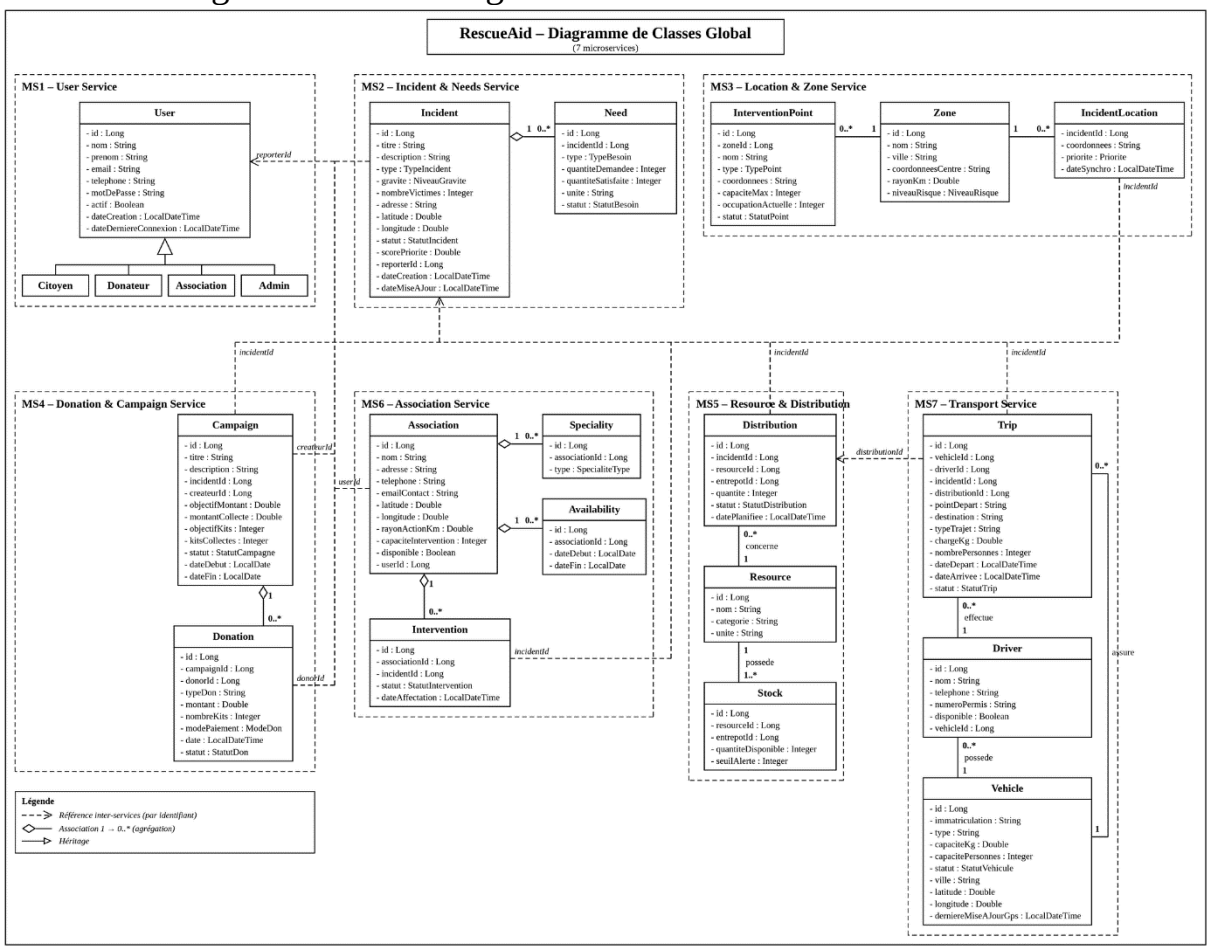
10 Conception

1.24 Diagramme des cas d’utilisations

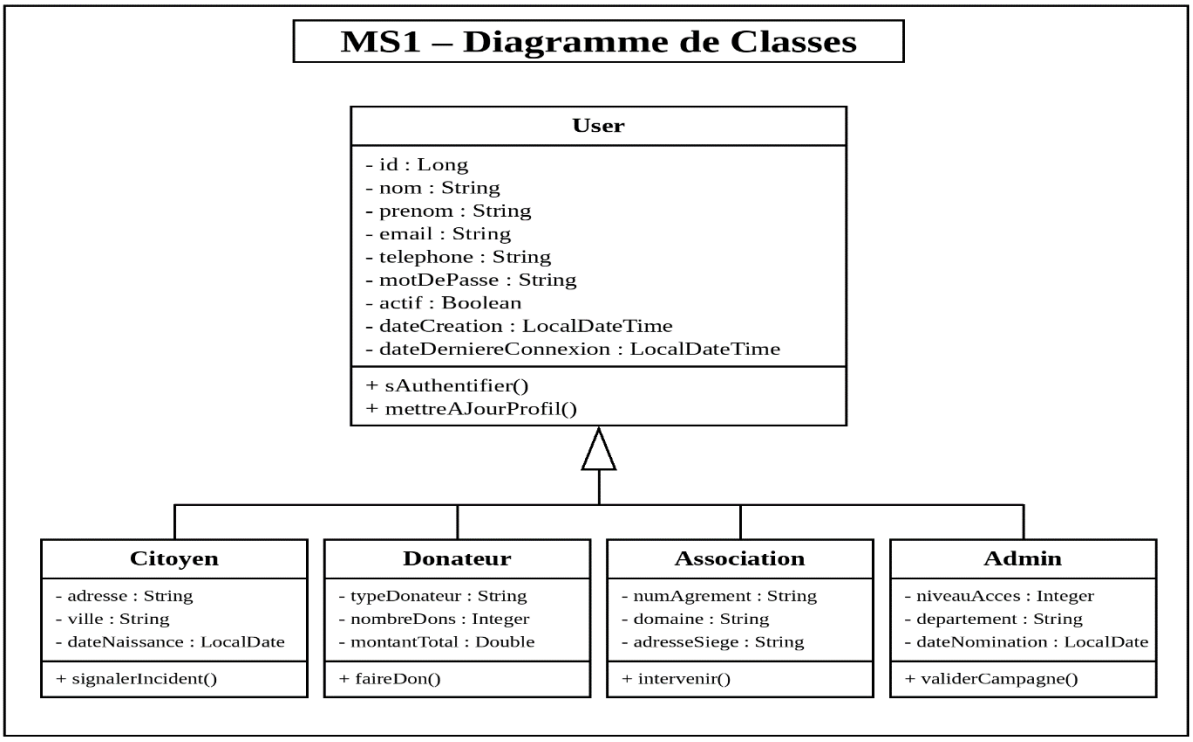


1.25 Diagrammes de classes

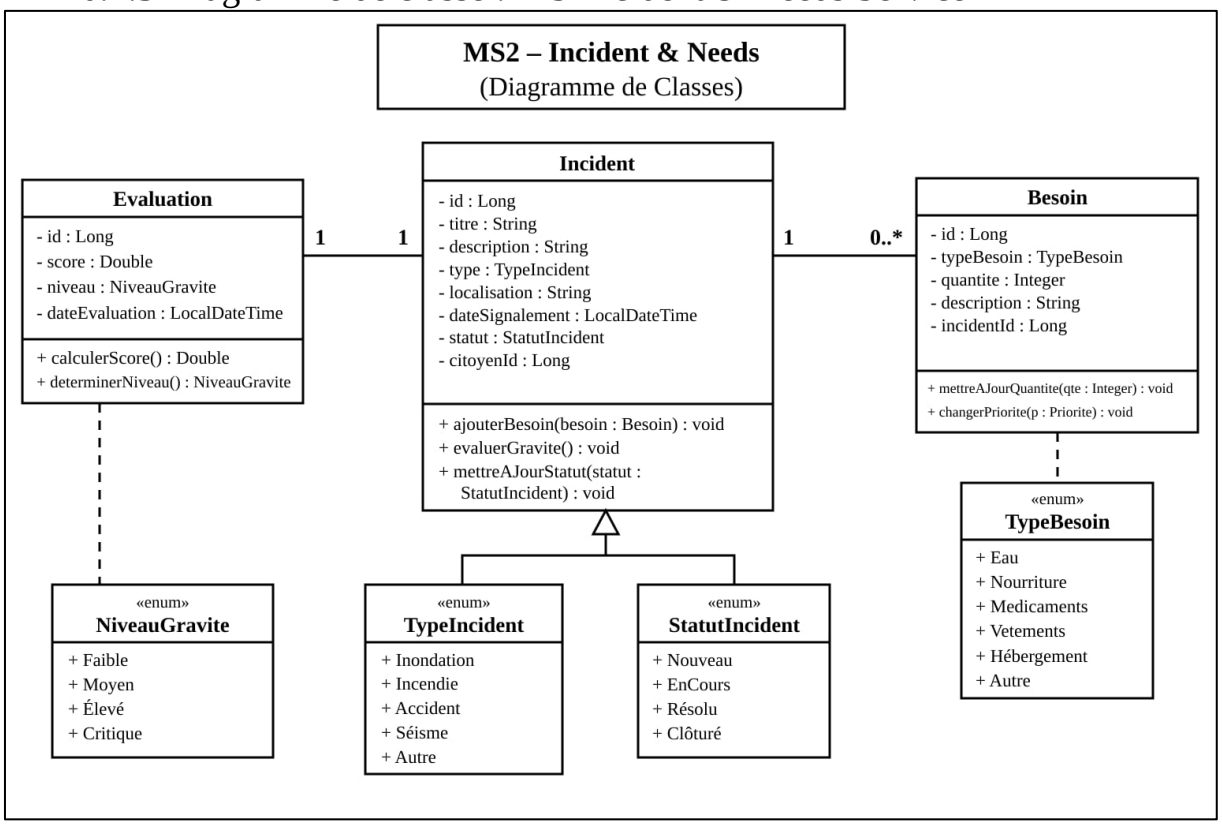
10.1.1 Diagramme de classe globale



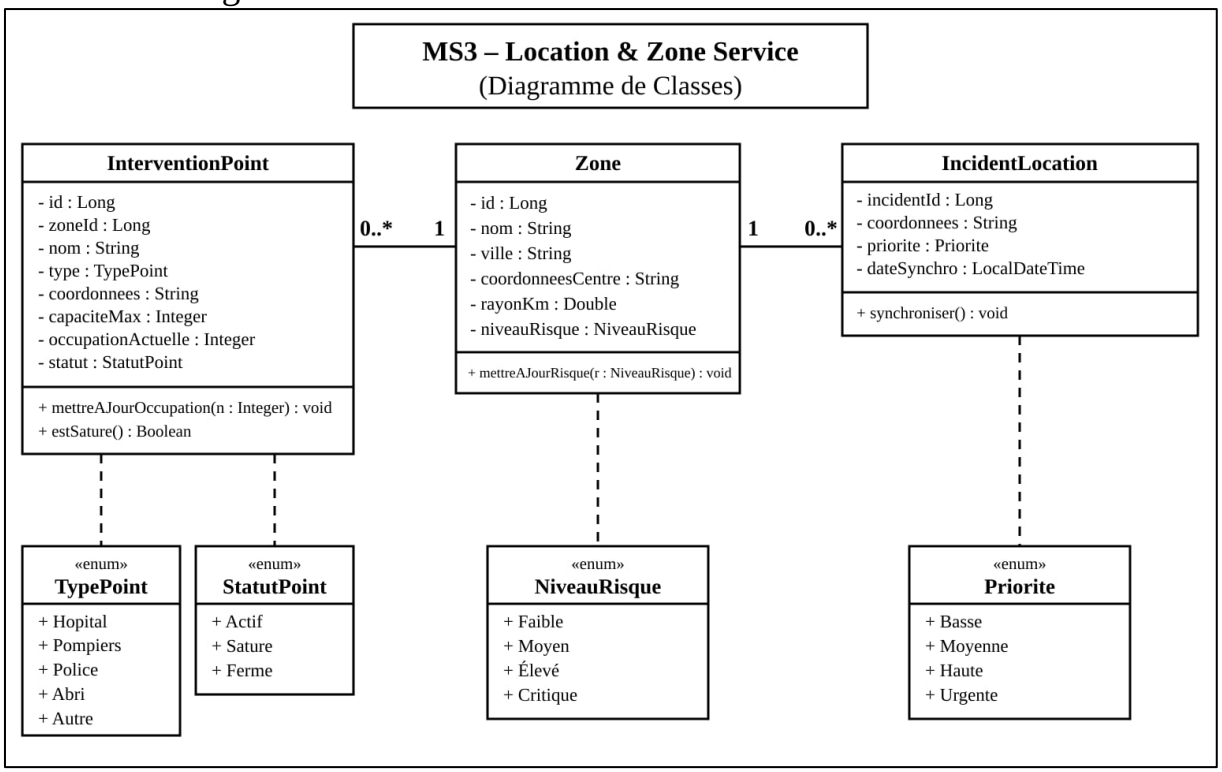
10.1.2 Diagramme de classe : MS1-User Service



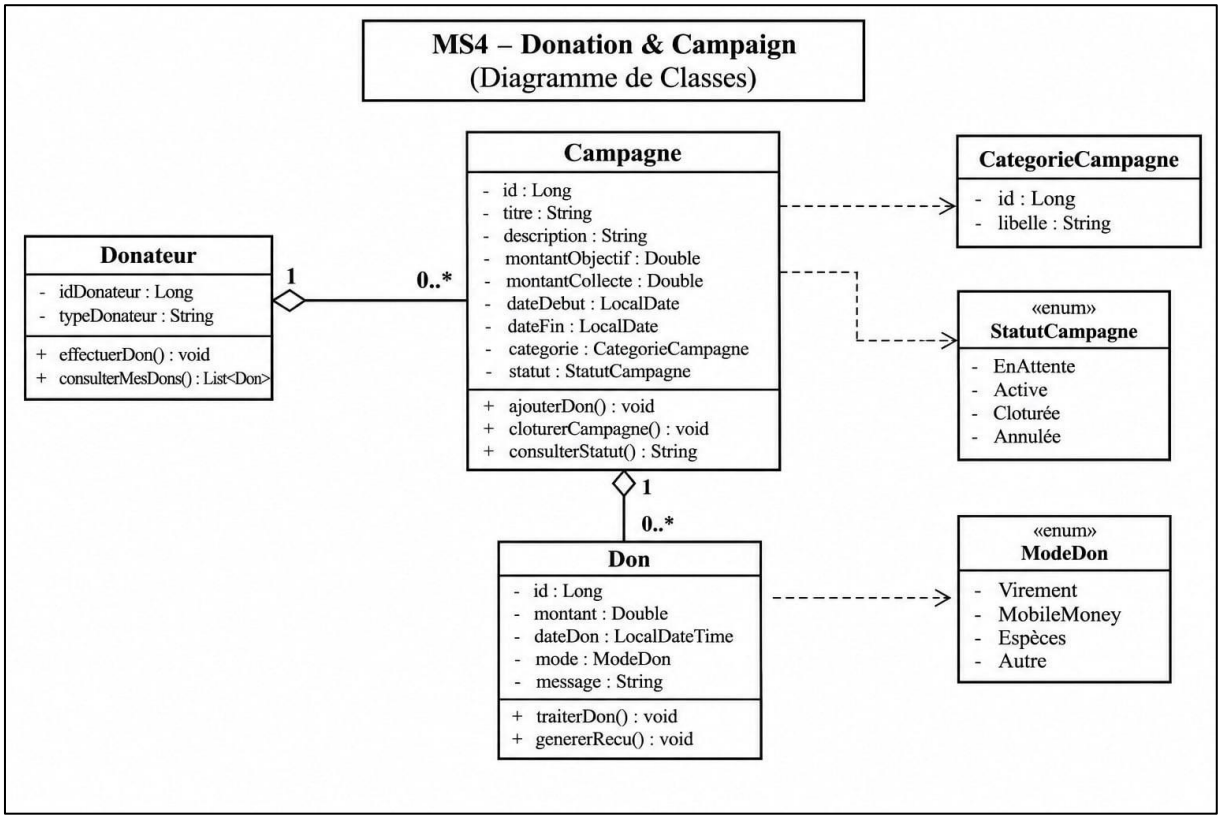
10.1.3 Diagramme de classe : MS2-Icident & Needs Service



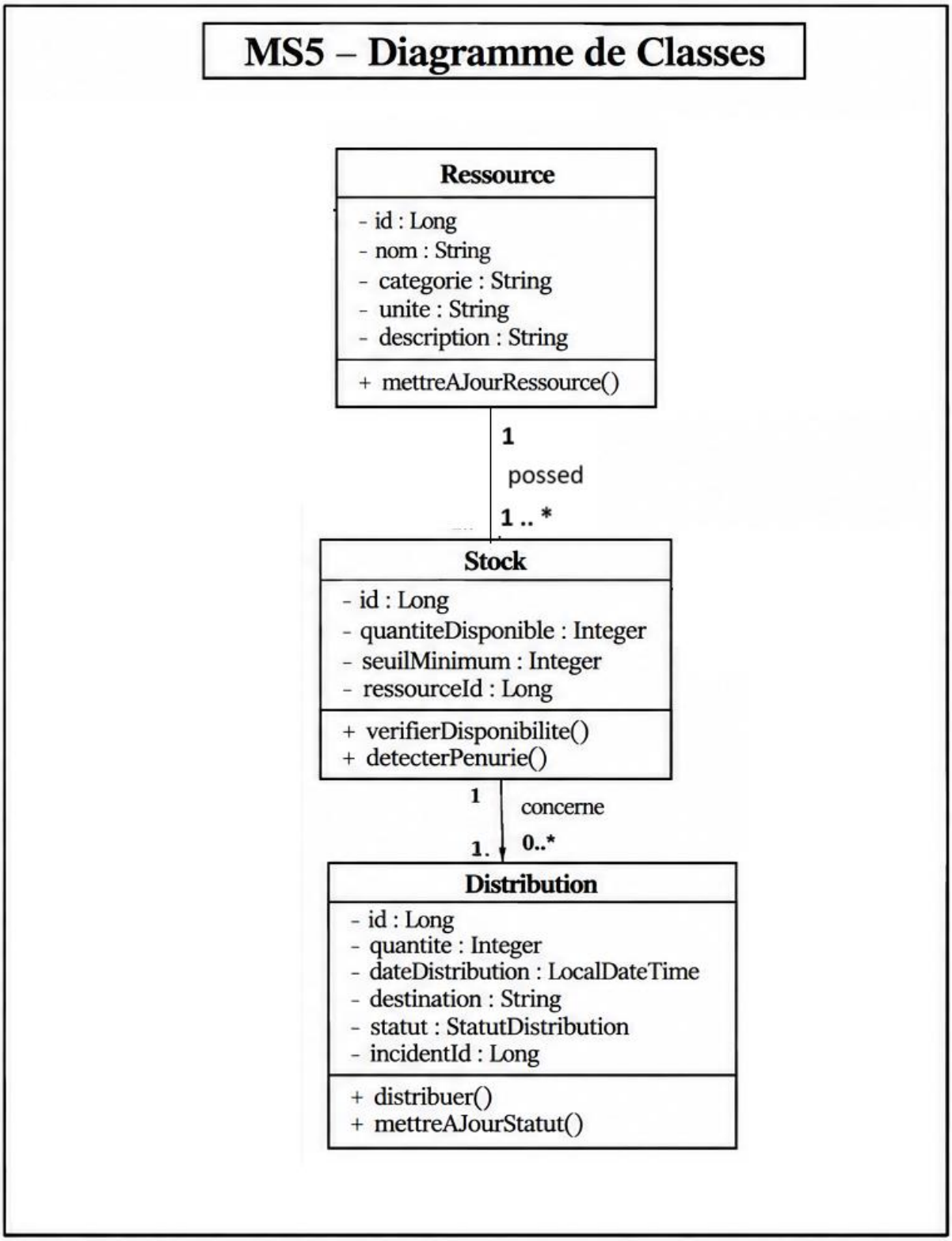
10.1.4 Diagramme de classe : MS3- Location & Zone Service



10.1.5 Di agramme de classe : MS4- Donation & Campaign Service

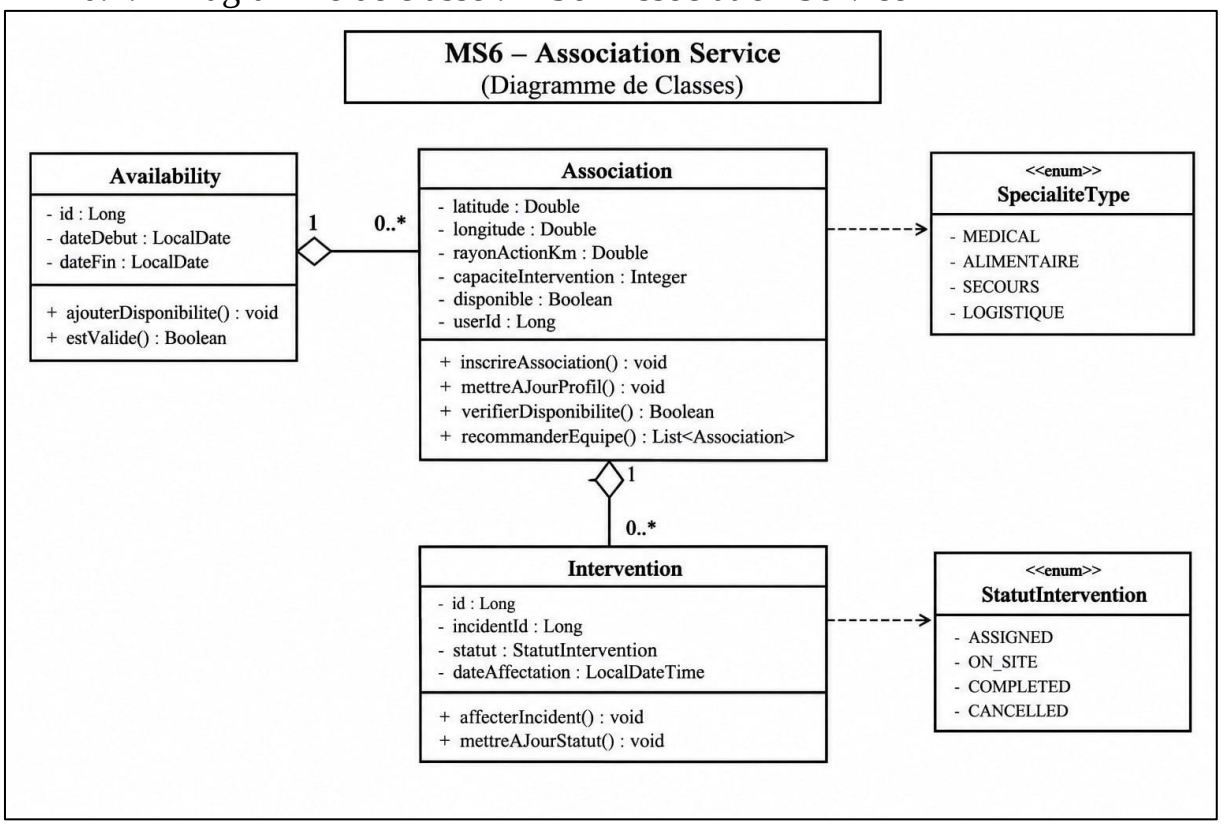


10.1.6 Diagramme de classe : MS5- Resource & Distribution Service

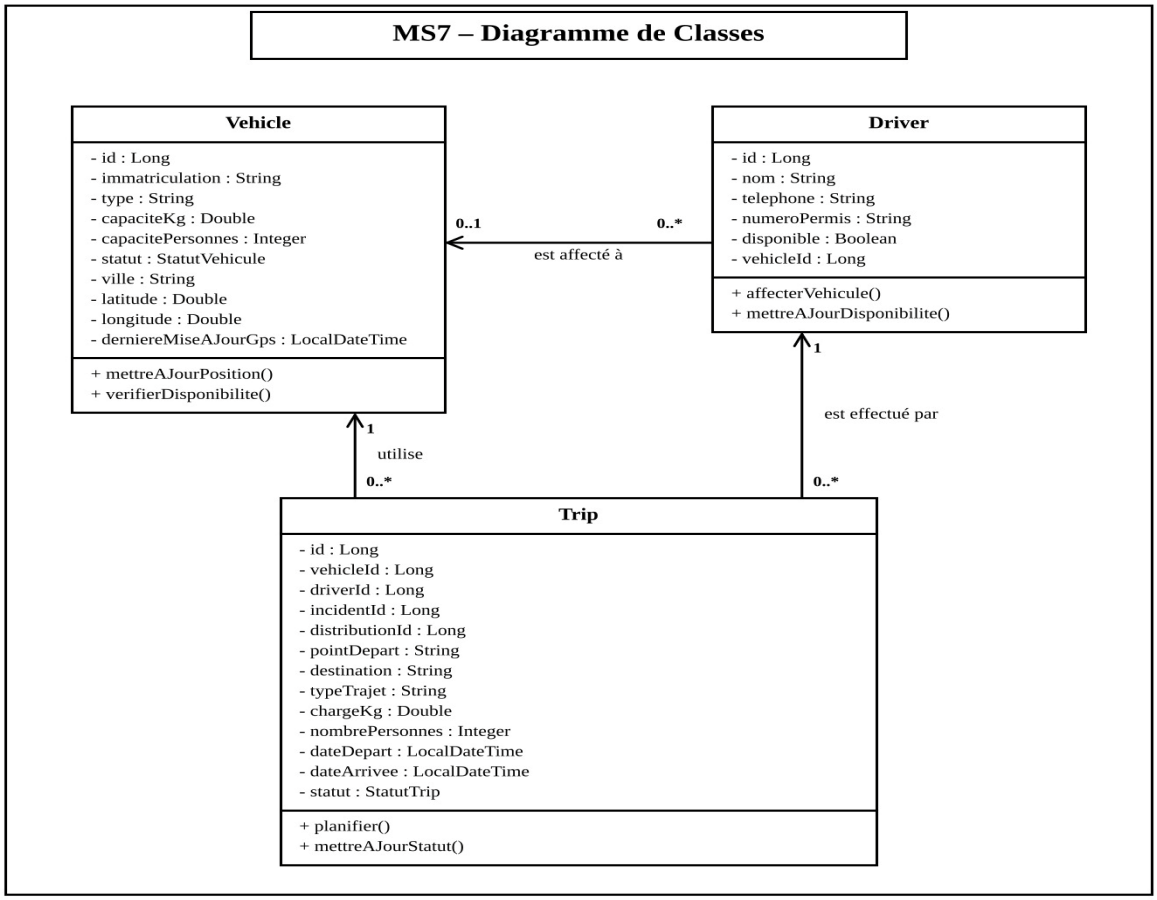


s

10.1.7 Diagramme de classe : MS6- Association Service



10.1.8 Diagramme de classe : MS7- Transport Service



11 Conclusion

RescueAid propose une architecture microservices faiblement couplée, réalisable dans un cadre académique. Chaque service a des données et une responsabilité propres, un CRUD complet et une règle métier avancée démontrable. L'ensemble forme un scénario cohérent : du signalement d'un incident à la livraison de l'aide.